



Artificial and Computational Intelligence DSE ZG557

Contact Session 3

Problem Solving by Search

Focus: problem formulation, uninformed search and introductory heuristic search



BITS Pilani

Pilani Campus

Akhil P V | BITS Pilani WILP

Reference: T1 Chapter 3.1–3.4, 3.5.1–3.5.2

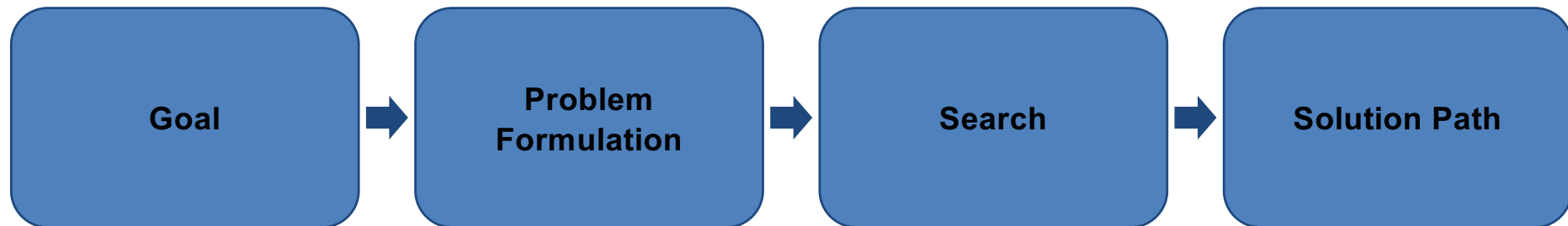
From Goals to Search: CS2 to CS3

CS2

A goal-based agent asks:
What future state do I want to reach?

CS3

A problem-solving agent asks:
What sequence of actions will take me from the current state to the goal state?



A goal states what to achieve. Problem formulation states what the agent may do, what follows from each action and what each action costs.

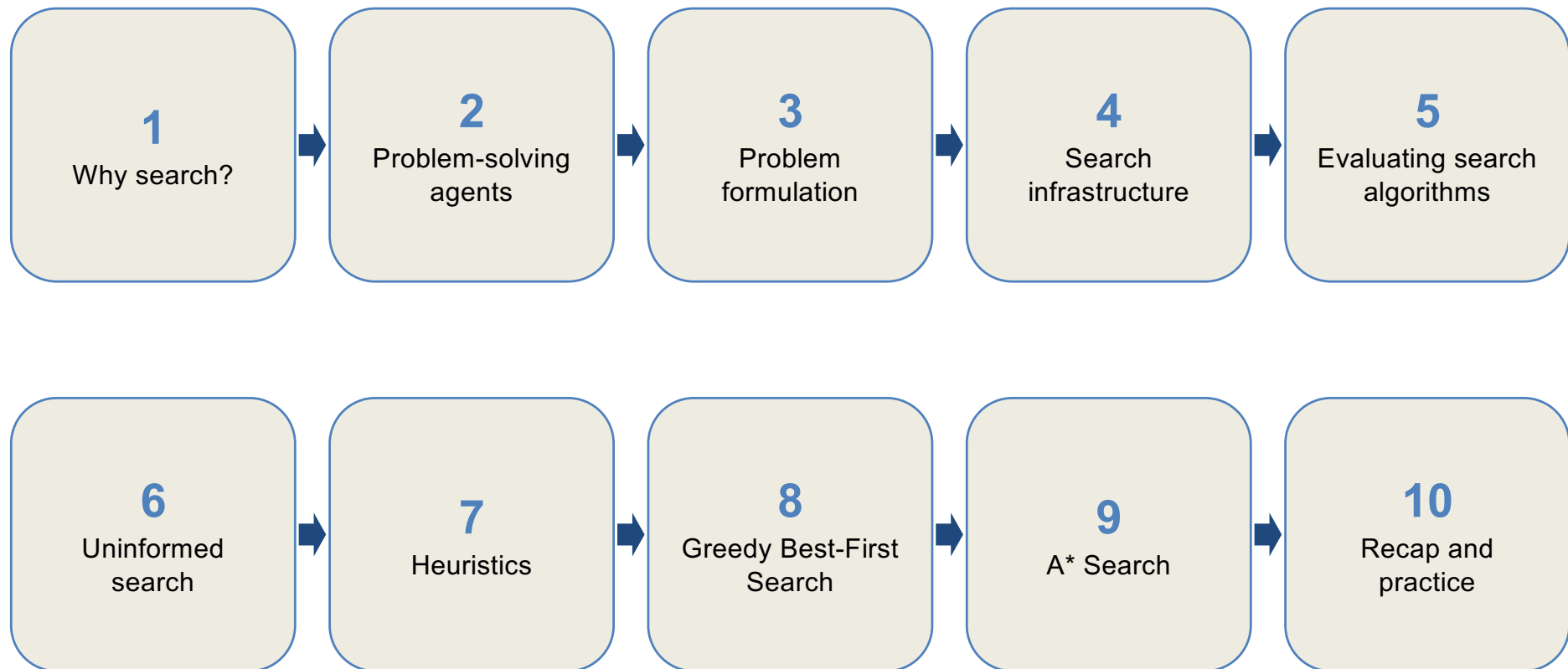
Learning Objectives



By the end of this session, you should be able to:

1. Explain the problem-solving-agent cycle.
2. Formulate a problem using five components.
3. Distinguish state space, search tree, state, node, frontier and explored set.
4. Compare search strategies using completeness, optimality, time and space.
5. Trace BFS, DFS, UCS, DLS and IDS.
6. Explain the role of a heuristic.
7. Trace Greedy Best-First Search and A* Search.
8. Compare UCS, Greedy and A*.

Today's Roadmap



Every strategy uses the same search framework; they differ in how the frontier is ordered.

T1, Chapter 3 structure.

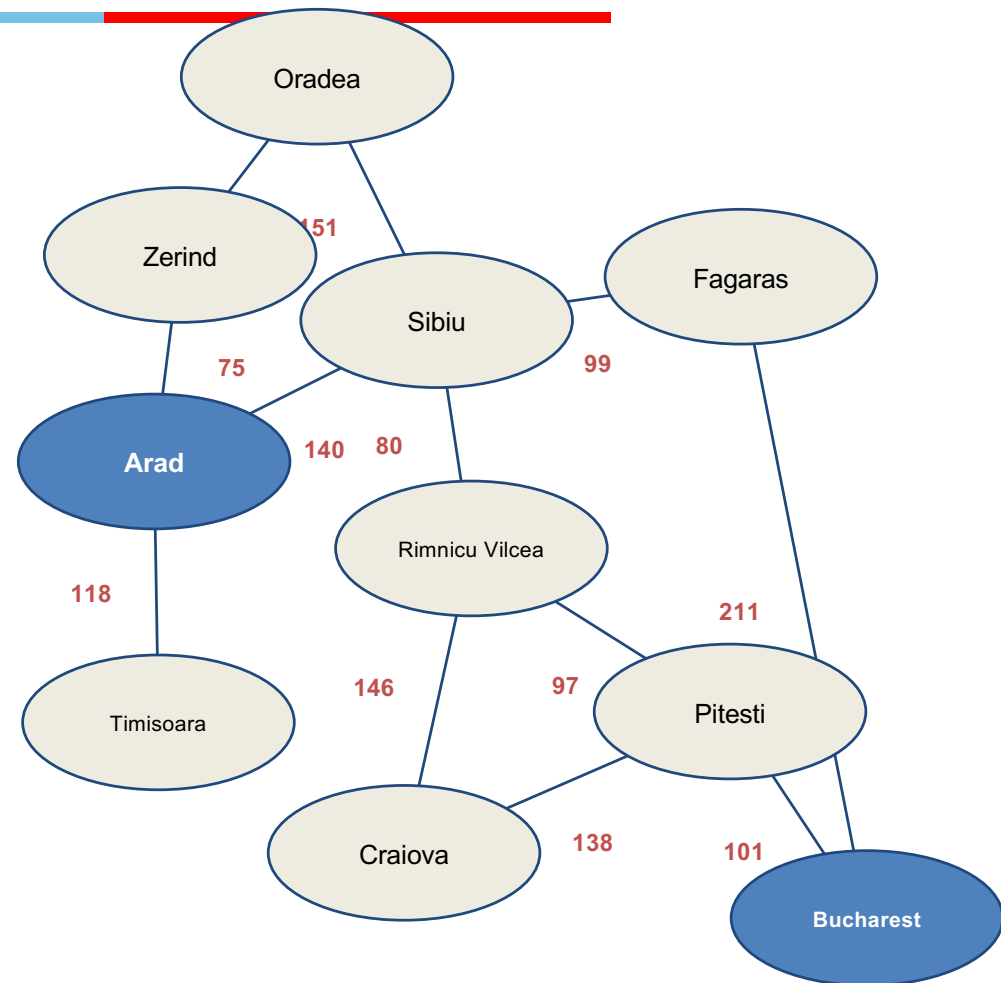
Motivating Example: Route Finding

Scenario

You are in Arad. You must reach Bucharest.

Possible questions

- Which cities can I travel to next?
- Which route reaches Bucharest?
- Which route is cheapest?
- How much search is required?
- Can a heuristic guide the search?



Arad = Start

Bucharest = Goal

T1, §3.2, simplified road map of part of Romania (schematic, not to scale).

Why Do Agents Need Search?

Immediate reaction

A simple reflex agent acts on the current percept.

Percept $\rightarrow$ Rule $\rightarrow$ Action

Reasoning over action sequences

A goal-based agent may need to ask which action sequence reaches the goal.

State $\rightarrow a_1, a_2, \dots, a_k \rightarrow$ Goal

route planning

8-puzzle

robot navigation

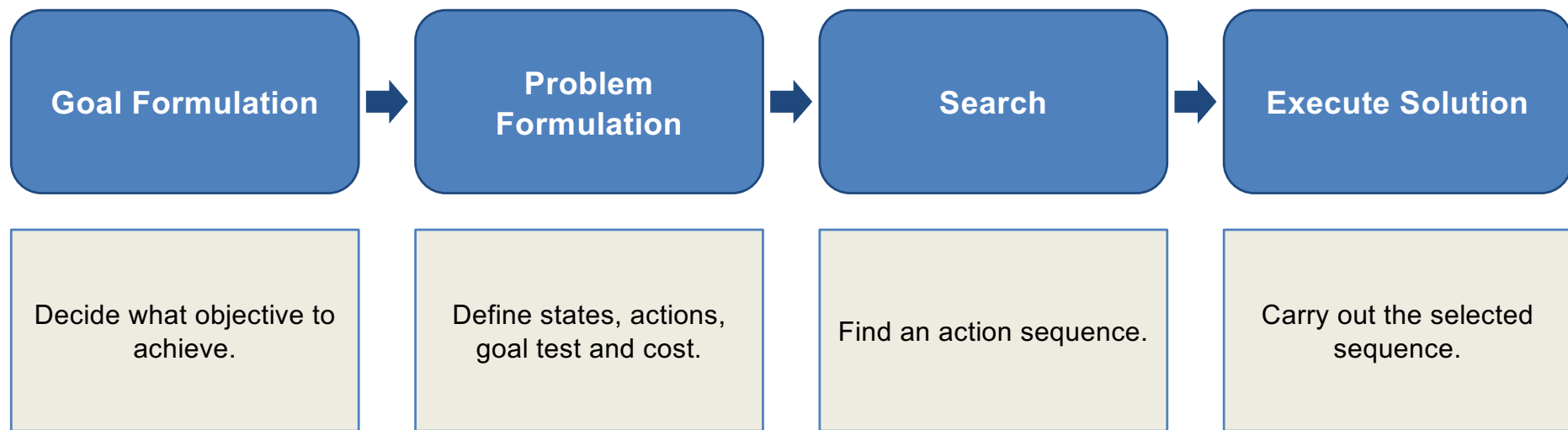
delivery
scheduling

maze solving

Search

Systematic exploration of possible action sequences.

The Problem-Solving Agent Cycle



Search is one stage of problem solving

Goal formulation limits what must be considered; problem formulation creates the abstract search problem that the search algorithm then solves.

Assumptions in Classical Search

Classical search usually assumes the environment is:

observable

known

discrete

deterministic

static

single-agent or non-
adversarial

Under these assumptions, a solution can be
a fixed sequence of actions

Five Components of a Well-Defined Problem



A search problem is specified by
⟨Initial State, Actions, Result, Goal Test, Path Cost⟩

Component	Meaning
Initial state	Where the agent starts
Actions	What the agent can do
Result / transition model	What each action leads to
Goal test	Whether the goal has been reached
Path cost	Total cost of an action sequence

Route-Finding Formulation

Problem: go from Arad to Bucharest.

Component	Route-finding example
Initial state	In(Arad)
Actions	Go to a neighbouring city
Result	Result(In(Arad), Go(Sibiu)) = In(Sibiu)
Goal test	Is current state In(Bucharest)?
Path cost	Sum of road distances

A possible solution: Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucharest

Abstraction in Problem Formulation

A real-world route contains many details. Formulation decides what to model.

Ignore

- radio volume
- colour of buildings
- exact lane changes
- weather details, if not modelled
- sightseeing preferences

Retain

- cities
- roads
- distances
- start state
- goal state

Good abstraction keeps what matters for solving the problem.

8-Puzzle Formulation

Component	Description
Initial state	Any arrangement of 8 tiles and one blank
Actions	Move blank left, right, up or down
Result	New tile configuration
Goal test	Configuration matches the goal arrangement
Path cost	Usually 1 per move

Reachable states: $9! / 2 = 181,440$

Initial state

7	2	4
5		6
8	3	1

Goal state

	1	2
3	4	5
6	7	8

Tiles are numbered; one square is blank.

Vacuum World as a Search Problem

Two-location vacuum world

State includes:

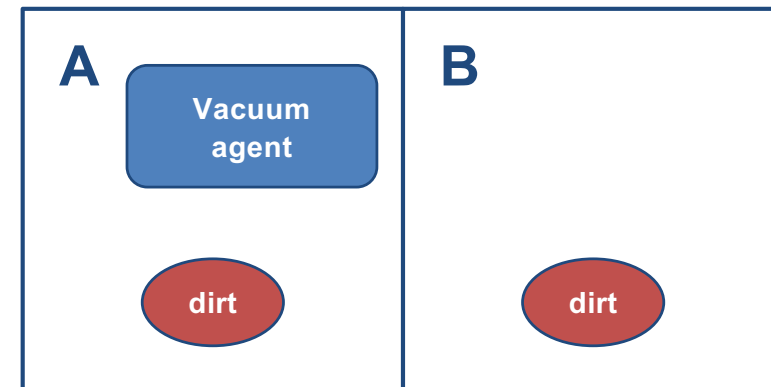
- agent location: A or B
- whether A is dirty
- whether B is dirty

Total states: $2 \times 2^2 = 8$

Actions: {Left, Right, Suck}

Goal test: A clean and B clean

Path cost: 1 per action



State shown: agent in A, both A and B dirty.

State Space



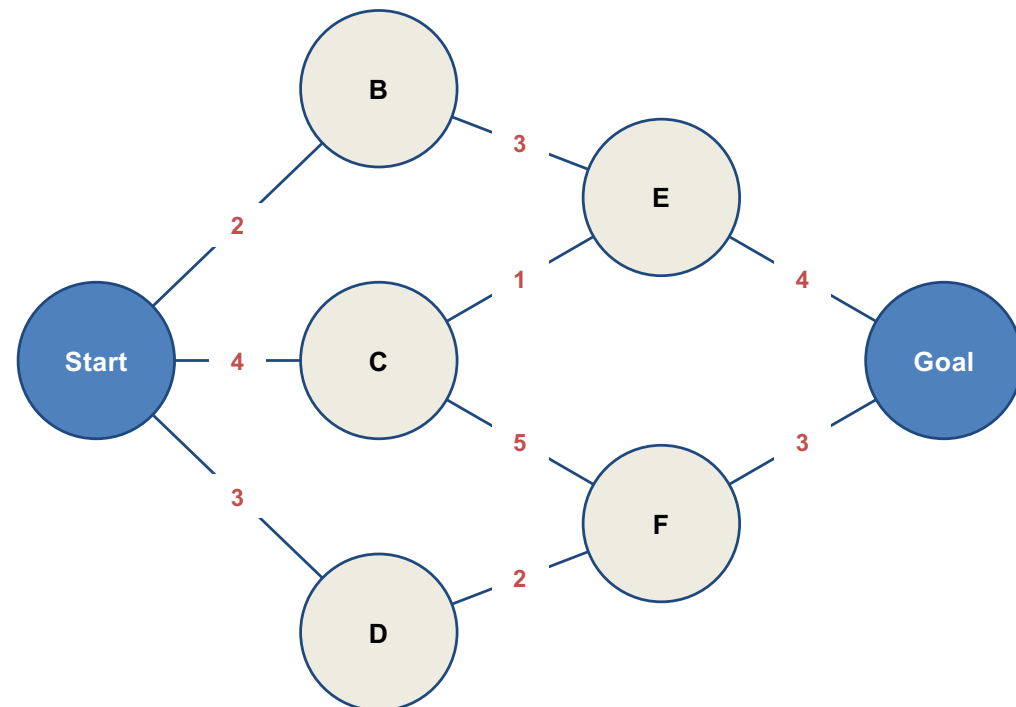
State space

The set of all states reachable from the initial state by applying actions.

It forms a graph:

- nodes = states
- edges = actions
- edge labels = action costs

A solution is a path from the initial state to a goal state.

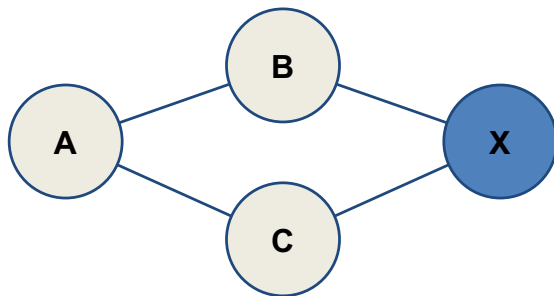


Alternative branches give alternative paths to the goal.

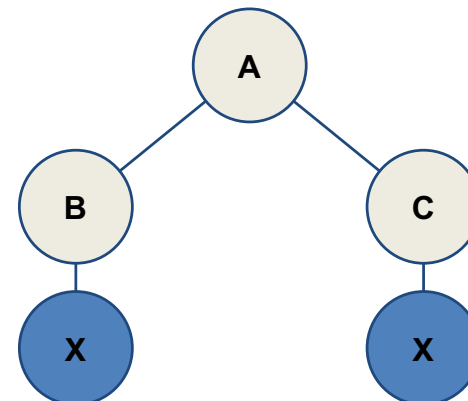
Search Tree vs State-Space Graph

State-space graph	Search tree
Represents unique states and actions	Represents generated paths
Same state appears once	Same state may appear multiple times
Compact	May contain repeated states
Describes the problem	Describes the search process

State-space graph: X reached by two routes



Search tree: X appears as two separate nodes



Key idea: State ≠ Node

T1, §3.3, Searching for Solutions.



What Information Is Stored in a Node?

STATE

PARENT

ACTION

PATH-COST

Field	Meaning
STATE	Current problem state
PARENT	Node that generated this node
ACTION	Action applied to the parent
PATH-COST $g(n)$	Cost from the initial node to this node

Parent pointers allow the solution path to be reconstructed once a goal node is found.

Frontier and Explored Set

Frontier

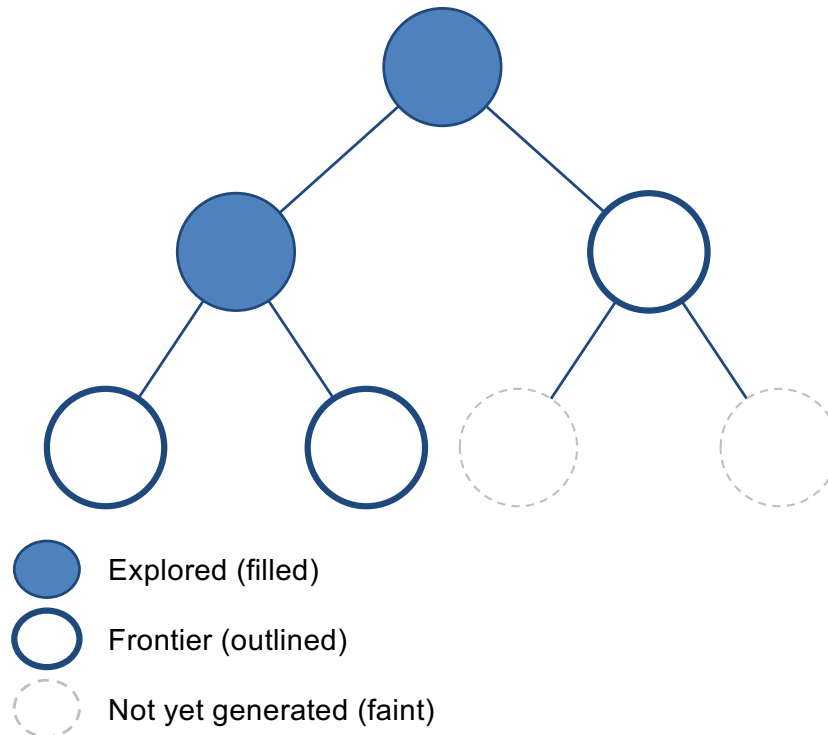
Generated nodes waiting to be expanded.

Explored set

States already expanded.

Expansion

Taking a node and generating its successors.



Choose Node



Goal Test



Expand



Update Frontier

Different algorithms mainly differ in which frontier node they choose next.

Generic Graph Search Skeleton

```
initialise frontier with the initial node
initialise explored set as empty

loop:
  if frontier is empty:
    return failure

  choose a node from frontier

  if node satisfies goal test:
    return solution

  add node.STATE to explored
  expand node
  add valid successors to frontier
```

The strategy is decided by:
**Which node is chosen
from the frontier?**

BFS, DFS, UCS, Greedy and A* all
follow this skeleton.
**Only the frontier-selection rule
changes.**

Evaluating Search Algorithms

We compare search strategies using four criteria:

Criterion	Question
Completeness	Will it find a solution if one exists?
Optimality	Will it find the least-cost solution?
Time complexity	How many nodes are generated or expanded?
Space complexity	How many nodes are stored?

Common notation

b branching factor	d depth of shallowest goal	m maximum depth	ℓ depth limit
------------------------------	--------------------------------------	---------------------------	-------------------------

T1, §3.3.2, Measuring problem-solving performance.

One Framework for All Search Strategies



Strategy	Frontier-selection rule
BFS	Expand shallowest node
DFS	Expand deepest node
UCS	Expand lowest $g(n)$
DLS	DFS subject to depth limit ℓ
IDS	Repeated DLS with increasing ℓ
Greedy	Expand lowest $h(n)$
A*	Expand lowest $g(n) + h(n)$

Search algorithms differ mainly in how they order the frontier.

Uninformed Search

Uses only the problem definition:

- initial state
- actions
- transition model
- goal test
- path cost

No heuristic guidance

Uninformed search does not use additional information about which non-goal state appears closer to the goal.

All strategies below share one search framework and differ only in how the frontier is ordered.

BFS

Shallowest first

DFS

Deepest first

UCS

Lowest $g(n)$ first

DLS

Depth limit ℓ

IDS

Increasing limits

Uninformed does not mean random: it means no problem-specific goal-distance estimate.

Breadth-First Search

Rule

Expand the shallowest node first.

Frontier

FIFO queue

Behaviour

Explore the search tree level by level.

Properties

- complete if the branching factor is finite
- optimal if all step costs are equal
- high memory usage

Depth 0



Depth 1



Depth 2



BFS sweeps each depth completely before going deeper.

BFS Trace



Given

$A \rightarrow \{B, C\}$

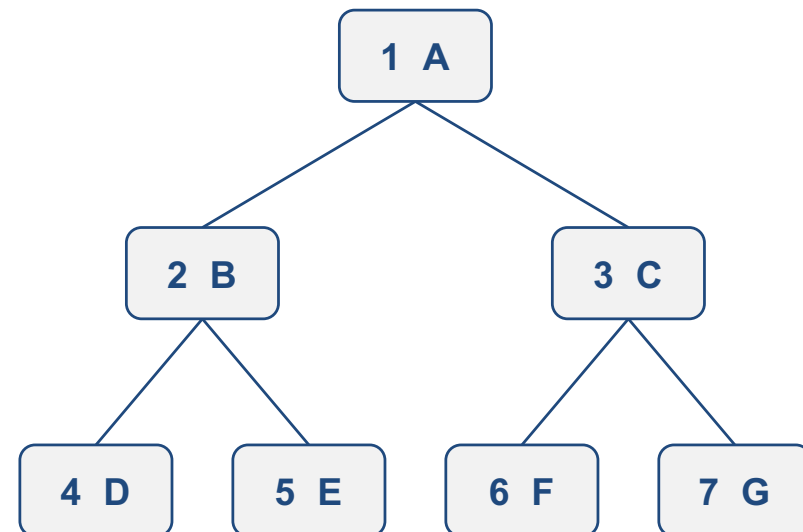
$B \rightarrow \{D, E\}$

$C \rightarrow \{F, G\}$

BFS expansion order

A, B, C, D, E, F, G

Frontier behaviour: FIFO



Numbers show the BFS expansion order.

Depth-First Search

Rule

Expand the deepest node first.

Frontier

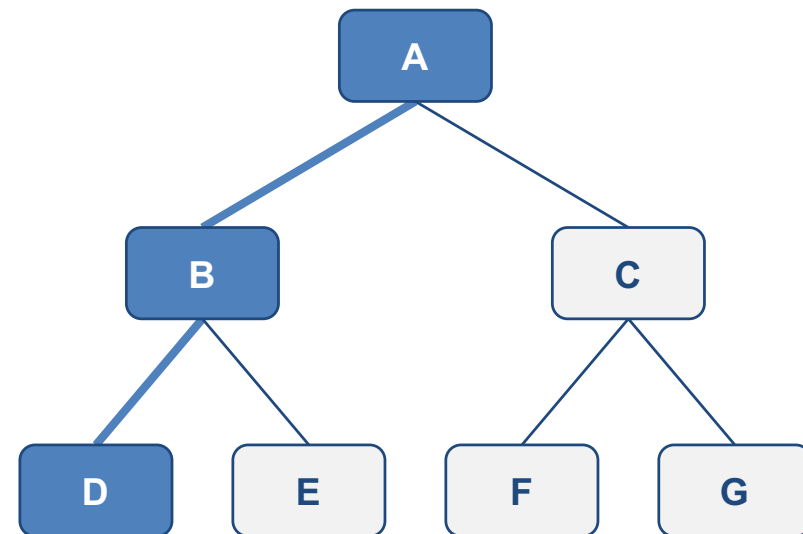
LIFO stack

Behaviour

Follow one branch deeply, then backtrack.

Properties

- low memory
- not optimal
- not complete in infinite-depth spaces



DFS drives down $A \rightarrow B \rightarrow D$ before backtracking.

DFS Trace



Given the same tree

$A \rightarrow \{B, C\}$

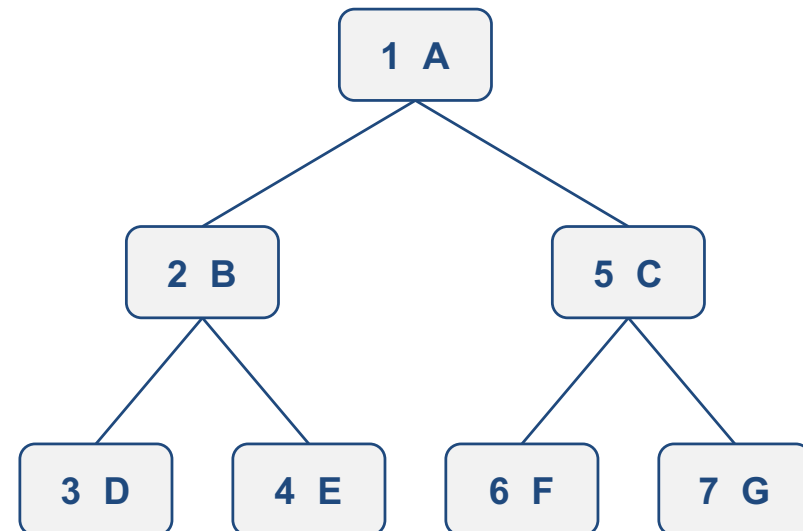
$B \rightarrow \{D, E\}$

$C \rightarrow \{F, G\}$

Assuming left-first expansion

A, B, D, E, C, F, G

Frontier behaviour: LIFO



Numbers show the DFS expansion order.



BFS vs DFS, and the Lab 1 Connection

Feature	BFS	DFS
Frontier	Queue (FIFO)	Stack (LIFO)
Main idea	Level by level	Go deep
Complete?	Yes, with finite branching	No for infinite-depth spaces
Optimal?	Yes for unit step cost	No
Memory	High	Low

Lab 1: implement BFS and DFS — the frontier data structure is the only structural difference.

BFS → queue DFS → stack or recursion

T1, §3.4.1 and §3.4.3; DSE ZG557 Lab 1.



Uniform-Cost Search

Rule

Expand the node with the lowest path cost.

Frontier

Priority queue ordered by $g(n)$

$g(n)$

cost from the start node to node n

Use case

When step costs are not equal, the shallowest path is not necessarily the cheapest path.

BFS optimises the number of steps when every step costs the same.

UCS optimises total path cost.

UCS expands the cheapest path-so-far first.



UCS: Why Not Stop at the First Generated Goal?

In UCS, a generated goal node may not yet be optimal.

Correct rule

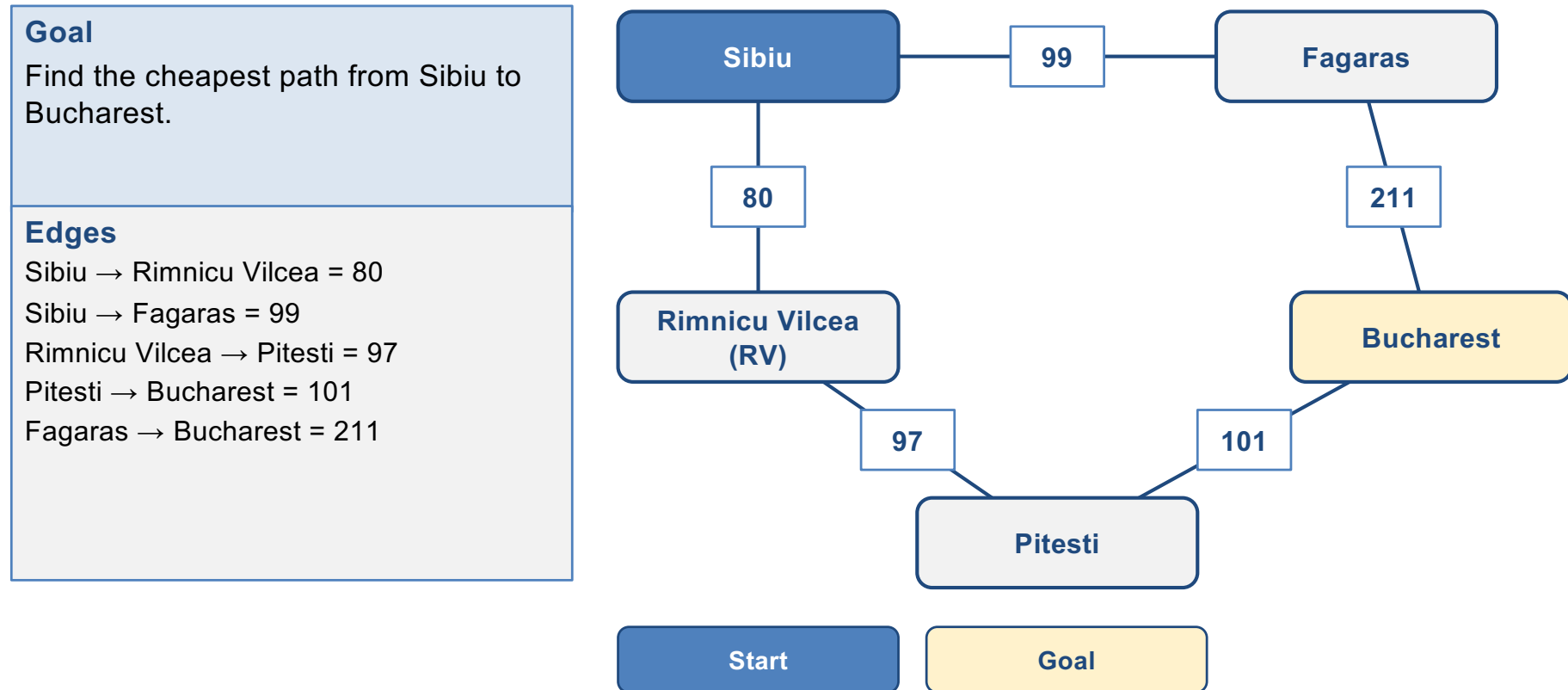
Return the solution when the goal node is selected for expansion from the priority queue.

Reason

Another frontier path with lower cost may still reach the same goal.

First generated goal $\neq$ best goal

UCS Worked Trace: Setup



T1, §3.4.2; route distances after T1, Figure 3.2.

UCS Worked Trace



Step	Expand	Frontier ordered by g(n)
0	—	Sibiu: 0
1	Sibiu	RV: 80, Fagaras: 99
2	RV	Fagaras: 99, Pitesti: 177
3	Fagaras	Pitesti: 177, Bucharest: 310
4	Pitesti	Bucharest: 278
5	Bucharest	Return solution

Solution: Sibiu → Rimnicu Vilcea → Pitesti → Bucharest

Cost: $80 + 97 + 101 = 278$

Depth-Limited Search

DLS = DFS + depth limit ℓ

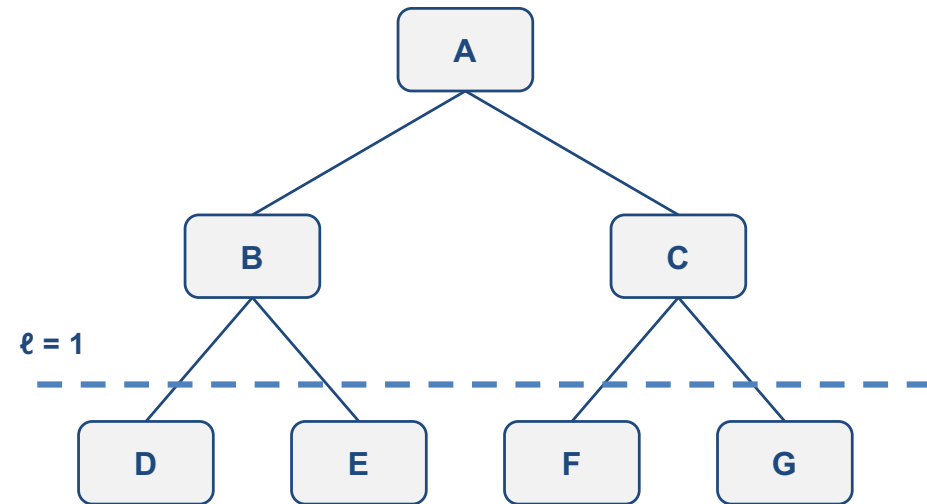
Purpose

Avoid exploring indefinitely deep paths.

Possible outcomes

- success
- failure
- cutoff

Problem: if $\ell < d$, the goal may not be reached.



Depth limit $\ell = 1$: nodes below the dashed line are not expanded, so the search returns cutoff.

Failure vs Cutoff in DLS

Outcome	Meaning
Success	Goal found
Failure	No solution exists in the searched space
Cutoff	Search stopped because the depth limit was reached

Important

Cutoff does not mean that no solution exists.

It means the solution may be deeper than the current limit ℓ .

Iterative Deepening Search

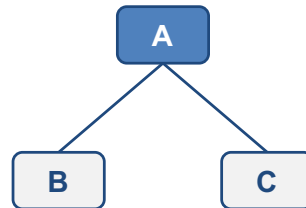
Repeatedly apply DLS with increasing limits: DLS(0), DLS(1), DLS(2), ... until a goal is found.

Combine BFS-like completeness with DFS-like low memory.

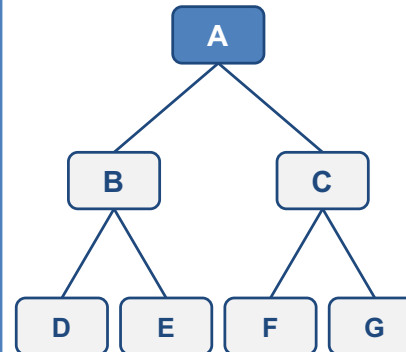
Depth limit 0



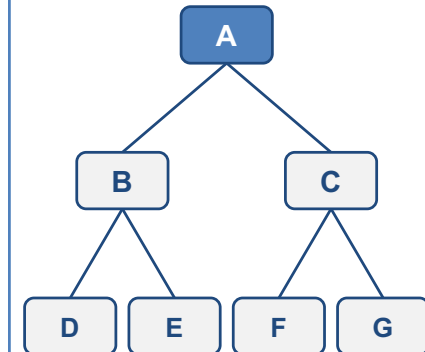
Depth limit 1



Depth limit 2



Depth limit 3



No deeper nodes exist here

Each iteration restarts from the root; only the depth limit changes.

IDS Properties



Property	IDS
Complete?	Yes, if the branching factor is finite
Optimal?	Yes, if step cost = 1
Time	$O(b^d)$
Space	$O(bd)$
Why useful? It avoids the large memory demand of BFS while retaining the discovery of shallow solutions.	
IDS repeats work but keeps memory linear in the depth.	



Comparing Uninformed Search Strategies

Algorithm	Frontier rule	Complete?	Optimal?
BFS	Shallowest first	Yes, finite branching	Yes, unit cost
DFS	Deepest first	No in infinite-depth spaces	No
UCS	Lowest $g(n)$	Yes, positive step costs	Yes
DLS	Deepest within limit ℓ	No if the limit is too small	No
IDS	Increasing depth limits	Yes	Yes, unit cost

Completeness and optimality belong to individual algorithms under stated assumptions.

T1, §3.4, Figure 3.21 comparison.

Why Do We Need Informed Search?

Uninformed search does not use problem-specific guidance about which non-goal state appears closer to the goal.

But knowledge is often available

In many problems the agent has extra information about the goal that is not part of the problem definition.

Example

From Arad, when the goal is Bucharest, the straight-line distance to Bucharest can guide the search.

Guidance = heuristic information

Heuristics let the search prefer promising directions.

The Heuristic Function

$h(n)$

estimated cost of the cheapest path from node n to a goal

For a goal node: $h(\text{goal}) = 0$

Example

$h(\text{Arad}) = 366$

366 is the straight-line distance from Arad to Bucharest.

A heuristic estimates the remaining cost; it is not the actual remaining road cost.

T1, §3.5, straight-line distances after T1, Figure 3.22.



Informed Search and the Evaluation Function

In best-first search the next node is chosen using an evaluation function $f(n)$.
The node with the lowest $f(n)$ is expanded first.

Algorithm	Evaluation function
Greedy Best-First Search	$f(n) = h(n)$
A* Search	$f(n) = g(n) + h(n)$

Different choices of $f(n)$ produce different strategies.

The search framework is unchanged; only the frontier priority changes.



Greedy Best-First Search

Rule

Expand the node estimated to be closest to the goal: $f(n) = h(n)$

Meaning

Ignore the cost already paid and choose the smallest estimated remaining cost.

Strength

Often moves quickly towards the goal and expands few nodes.

Weakness

May choose an expensive path, because $g(n)$ is ignored.

Greedy uses only the estimated remaining cost $h(n)$.

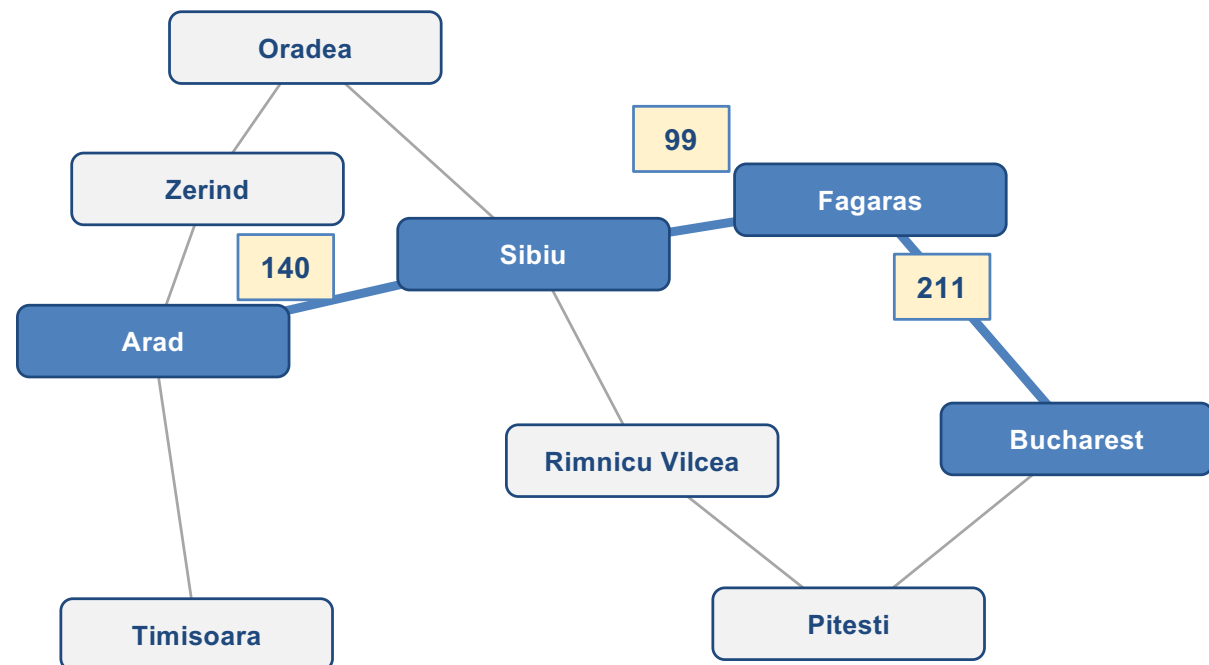
Greedy Trace: Arad to Bucharest

Straight-line heuristic to Bucharest

City	$h(n)$
Arad	366
Sibiu	253
Timisoara	329
Zerind	374
Fagaras	176
Rimnicu Vilcea	193
Bucharest	0

Greedy path

Arad → Sibiu → Fagaras → Bucharest



Path cost: $140 + 99 + 211 = 450$

T1, §3.5.1; distances after T1, Figures 3.2 and 3.22.

A* Search: Main Idea



A* balances cost so far and estimated cost remaining.

$$f(n) = g(n) + h(n)$$

$g(n)$

cost from the start node to n

$h(n)$

estimated cost from n to a goal

$f(n)$

estimated total cost of a solution
through n

A* estimates the total solution cost through each frontier node.

UCS, Greedy and A* Compared

Search	Uses $g(n)$?	Uses $h(n)$?	Evaluation
UCS	✓	—	$f(n) = g(n)$
Greedy	—	✓	$f(n) = h(n)$
A*	✓	✓	$f(n) = g(n) + h(n)$

Intuition

UCS

Cheapest so far

Greedy

Closest-looking

A*

Cheapest estimated complete path

T1, §3.4.2, §3.5.1 and §3.5.2.

A* Worked Trace: Setup

Task

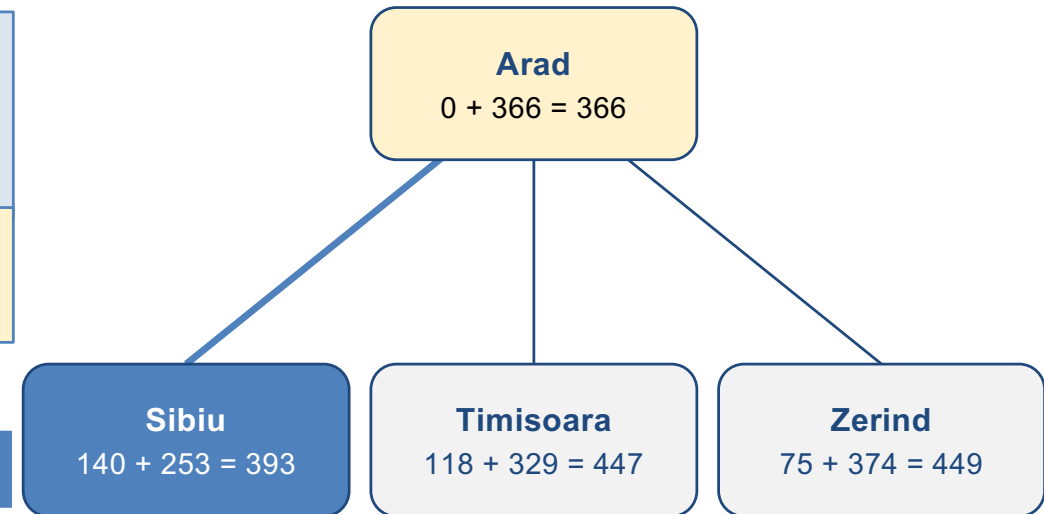
Find a route from Arad to Bucharest using $f(n) = g(n) + h(n)$.

Arad: $g = 0$, $h = 366$, $f = 366$

After expanding Arad

Node	$g(n)$	$h(n)$	$f(n)$
Sibiu	140	253	393
Timisoara	118	329	447
Zerind	75	374	449

Expand next: Sibiu



Each child is labelled $g + h = f$.

T1, §3.5.2; distances after T1, Figures 3.2 and 3.22.

A* Trace: Middle Steps

After expanding Sibiu

Node	g(n)	h(n)	f(n)
Rimnicu Vilcea	220	193	413
Fagaras	239	176	415
Timisoara	118	329	447
Zerind	75	374	449
Oradea	291	380	671

Expand next: Rimnicu Vilcea

After expanding Rimnicu Vilcea

Node	g(n)	h(n)	f(n)
Fagaras	239	176	415
Pitesti	317	100	417
Timisoara	118	329	447
Zerind	75	374	449
Craiova	366	160	526

Expand next: Fagaras (f = 415)

A* continuously reorders the frontier by $g(n) + h(n)$.

A* Trace: Reaching the Goal

After expanding Fagaras

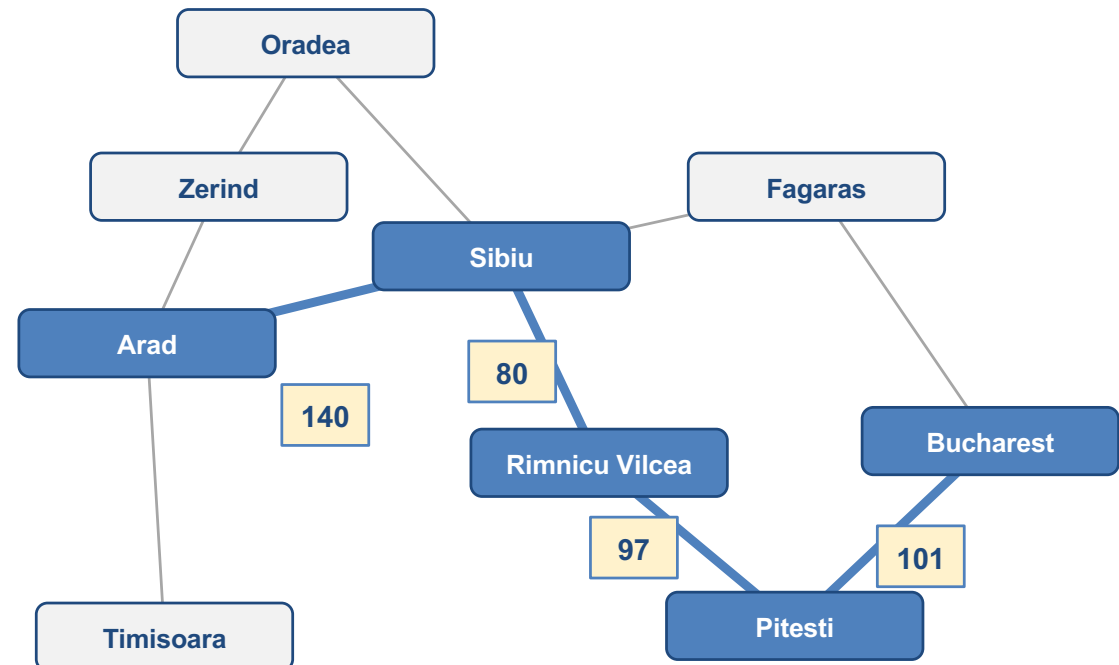
Node	g(n)	h(n)	f(n)
Pitesti	317	100	417
Timisoara	118	329	447
Zerind	75	374	449
Bucharest	450	0	450

Expand Pitesti

Then Bucharest: $g = 418$, $h = 0$, $f = 418$

Solution

Arad $\rightarrow$ Sibiu $\rightarrow$ Rimnicu Vilcea $\rightarrow$ Pitesti $\rightarrow$ Bucharest



Path cost: $140 + 80 + 97 + 101 = 418$

T1, §3.5.2; distances after T1, Figures 3.2 and 3.22.

Greedy vs A*: Same Problem, Different Result

Algorithm	Selected route	Cost
Greedy	Arad → Sibiu → Fagaras → Bucharest	450
A*	Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucharest	418

Why the difference?

Greedy uses
 $f(n) = h(n)$

A* uses
 $f(n) = g(n) + h(n)$

A* is less shortsighted than Greedy.



What We Will Study in CS4

Today

A*: $f(n) = g(n) + h(n)$

Be ready to work with

- $g(n)$, $h(n)$ and $f(n)$
- manual frontier tracing

Next session

- When is A* guaranteed optimal?
- What is an admissible heuristic?
- What is a consistent heuristic?
- How does heuristic quality affect performance?
- How are good heuristics designed?

A good heuristic guides the search without misleading it.

CS3 introduces how A* operates; CS4 studies why and when it is optimal.

Activity: Apply A* from S to G

Task

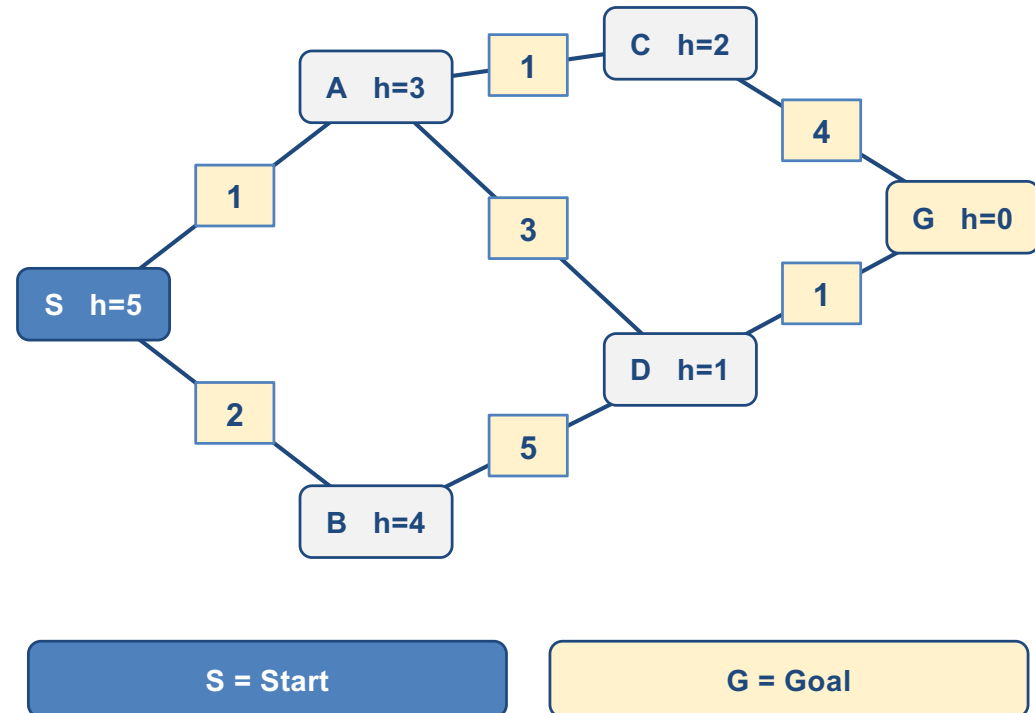
Trace the frontier using $f(n) = g(n) + h(n)$.

Heuristic values

Node	h
S	5
A	3
B	4
C	2
D	1
G	0

Edges

S-A = 1, S-B = 2, A-C = 1, A-D = 3,
B-D = 5, C-G = 4, D-G = 1



Common Mistakes in Search

- | | |
|---|--------------------------------------|
| 1. Confusing state with node. | 2. Forgetting path cost $g(n)$. |
| 3. Stopping UCS when a goal is first generated. | 4. Treating Greedy as optimal. |
| 5. Mixing up queue, stack and priority queue. | 6. Ignoring repeated states. |
| 7. Treating $h(n)$ as exact remaining cost. | 8. Using the A* formula incorrectly. |

Correct memory aid

$$\text{UCS} = g$$

$$\text{Greedy} = h$$

$$A^* = g + h$$

CS3 Summary



1. A problem-solving agent searches for an action sequence.

3. Search explores a state space using a frontier.

5. BFS and DFS support Lab 1.

7. DLS adds a depth limit; IDS repeats DLS.

9. A* expands lowest $g(n) + h(n)$.

2. A well-defined problem has five components.

4. Search strategies differ through frontier ordering.

6. UCS expands lowest $g(n)$.

8. Greedy expands lowest $h(n)$.

Search is systematic exploration controlled by a frontier-selection rule.

Exit Check



1. What are the five components of a well-defined problem?

2. How is a state different from a node?

3. What data structure does BFS use?

4. Why is DFS not always complete?

5. Why does UCS expand according to $g(n)$?

6. What is a heuristic?

7. How does A* differ from Greedy Best-First Search?

Answer briefly, in your own words.

Preparation for Contact Session 4

Next session: A* Optimality and Heuristic Quality

Read

- T1 §3.5.2 again
- T1 §3.6

Practice before next class

Trace A* on at least two small graphs.

Focus on

- admissible heuristics
- consistent heuristics
- dominance
- relaxed problems
- pattern databases
- effective branching factor

CS4 studies the quality and optimality conditions behind heuristic search.

T1, §3.5.2 and §3.6.

References



Primary course source

BITS Pilani WILP, Digital Learning Handout: Artificial and Computational Intelligence, Course No. DSE ZG557.

Prescribed textbook

Stuart Russell and Peter Norvig, Artificial Intelligence: A Modern Approach, Third Edition.

Sections used

- §3.1 Problem-Solving Agents
- §3.2 Example Problems
- §3.3 Searching for Solutions
- §3.4 Uninformed Search Strategies
- §3.5.1 Greedy Best-First Search
- §3.5.2 A* Search

Lab connection

Lab 1 — BFS and DFS implementation